

Web Programming Course

Exam - 6 June 2025

- This is an open-book exam, but all work must be done individually.
- Answer each question clearly, using sketches, tables, or descriptions where necessary.
- Do not write actual code, but provide detailed API designs, request/response formats, and data models.
- Write your name, surname, and student ID on the sheet where you answer the exam.

System for Managing Object Loans Between Users

We are required to design a web application that allows users to lend and borrow objects (books, tools, video games, etc.) in a trackable way.

Application Requirements

The system must include:

- An API to register a new object for lending (id, name, description, category, availability, owner id);
- An API to view the list of available objects;
- An API to request the loan of an object (object id, requester id, start date, expected duration);
- An API to view the loan history of an object (loans made or received).

IDs must be assigned by the server when an object is registered. The user IDs (owners and borrowers) are numeric. The availability field indicates whether the object is available or not (not the quantity). To define the APIs, assume that the loan is automatically accepted without checking availability.

Question 1: API Definition (15 points)

For each API:

- **3 points per API:** Request/response data models with types and constraints (e.g., max, min). Explain the reason for any constraints.
- **2 points per API:** Endpoint description (URL + HTTP method).
- **1 point per API:** Concrete example of a successful request and server response. Indicate the response code and any JSON response.

Question 2: Error Handling (5 points)

- **1 point total:** Explain the most relevant error codes (400, 404, 500).
- **2 points per example, max 4 points:** Provide two examples of requests that can cause one of the above errors in the previously designed APIs.

Question 3: Concurrency (3 points)

- **3 points total:** The system must prevent the same object from being loaned simultaneously to multiple users. How could a robust logic be designed for managing concurrency on shared objects?

Object Model

```
{
  "objectId": int,
  "name": string,
  "description": string,
  "category": string,
  "available": boolean,
  "ownerId": int
}
```

Constraints and Description:

- objectId: assigned incrementally by the server
- name: required, max 20 characters
- category: optional, predefined values (e.g., "book", "tool", "game")
- available: default true, changed when the object is loaned / accepted also integer with constraint > 0 or string with constraint of only two possible values
- ownerId: specifies the owning user

Loan Request Model

```
{
  "requestId": int,
  "objectId": int,
  "borrowerId": int,
  "startDate": datetime,
  "durationDays": int, min=1, max=30
}
```

Constraints and Description:

- requestId: identifies the request, assigned by the server
- objectId: must correspond to a registered object
- borrowerId: integer, specifies the user requesting the loan
- startDate: datetime, specifies when the loan starts
- durationDays: integer, max 30 days, min 1 day, specifies loan duration; accepted also datetime with check that date end > date start, and with a maximum duration check.

API 1: Register New Object

- Method: POST /objects
- Request:

```
{
  "name": "Classical Guitar",
  "description": "Good condition",
  "category": "tool",
  "ownerId": 2
}
```

The objectId and available fields are assigned by the server at the time of registration.

- **Response:** 201 Created

```
{
  "objectId": 1,
  "message": "Object registered successfully"
}
```

API 2: List Available Objects

- **Method:** GET /objects
- **Response:**

```
[
  {
    "objectId": 1,
    "name": "Classical Guitar",
    "description": "Good condition",
    "category": "tool",
    "ownerId": 2,
    "available": true
  }
]
```

API 3: Loan Request

- **Method:** POST /loans
- **Request:**

```
{
  "objectId": 1,
  "borrowerId": 4,
  "startDate": "2025-06-10",
  "durationDays": 7
}
```

- **Response:**

```
{
  "requestId": 15,
  "message": "Loan request approved."
}
```

This API changes the status of the identified object to available: false until the end of the loan.

API 4: Loan History for Object

- **Method:** GET /objects/1/loans
- **Response:**

```
[
  {
    "requestId": 15,
    "borrowerId": 4,
```

```
    "status": "accepted",
    "startDate": "2025-06-10",
    "durationDays": 7
  }
]
```

Error Handling

Relevant Error Codes:

- 400 Bad Request / 422 Unprocessable: missing or malformed data, for example a POST with an empty body, or some request that does not satisfy the data model
- 404 Not Found: the user requests a resource that does not exist, for example an object or loan not found
- 500 Internal Server Error: generic server error, for example a malfunction or unreachable database / connection

Error Example 1 – Malformed Request

Request:

```
{
  "objectId": 1,
  "borrowerId": "four",
  "startDate": "10/06/2025",
  "durationDays": 40
}
```

Response:

```
{
  "error": "Invalid data: User ID must be an integer, maximum
duration is 30 days"
}
```

Error Example 2 – Object Not Available

Request:

```
{
  "objectId": 100,
  "borrowerId": 4,
  "startDate": "2025-06-10",
  "durationDays": 5
}
```

Response:

```
{
  "error": "Non-existent object."
}
```

Question 3: Concurrency (3 points)

Concurrency can be managed through the “available” field and asynchronous handling of the requests.

Specifically, all requests will be put in a queue, and they are processed one at a time. Then, for each request, the logic flow first checks if the object is available, and if so, accepts the loan and

sets the field "available" as false. If not available, the request can optionally inform the user when the requested object will be available again.